Eric Huang

949-664-1648 | huangeric50@gmail.com | https://github.com/HuangEric22 | Personal Portfolio

Education

University of California, Los Angeles — Los Angeles, CA
Bachelor of Science in Computer Science, GPA: 3.71/4.0 — *Aug. 2022 – June 2026*

- Relevant Coursework: Neural Networks, Software Engineering, Computer Vision, Reinforcement Learning, Distributed Systems, Natural Language Processing, Programming Languages, Databases, Web Development

Technical Skills

Languages: C++, C, Python, SQL (Postgres), Java, JavaScript, Typescript, HTML/CSS, Bash, Go
Frameworks: React Native, Angular, FastAPI, Node.js, Flask, Express, Bootstrap, TensorFlow, PyTorch, CDK, SAM
Developer Tools: Git, Docker, MongoDB, WSL, VirtualBox, Firebase, AWS (Lambda, API Gateway, IAM), Agile, Jira
Libraries: Beautiful Soup, Selenium, Scikit-learn, Pandas, NumPy, Matplotlib, Keras, PyMongo, Three.js, Cobra, Jest

Experience

Software Engineer Intern | Amazon Web Services (EC2 Nitro) – *Seattle, WA* — Jun. 2025 – Sep. 2025

- Built and launched **Go** based CLI tool for onboarding, reducing onboarding time and manual workload by 75%
- Deployed **API Gateway** endpoints and AWS Lambda handlers with **SAM deploy**, allowing secure API calls
- Integrated **IAM roles** and **Midway cookies** for authentication, strengthening access control and security
- Automated cloud infrastructure deployment with **CloudFormation** and **CI/CD** cutting release time by 50%

LZR Research Assistant | Stanford Security Research – *Stanford, CA* — Oct. 2024 – Oct. 2025

- Optimized packet transmission and control flow using Go to speed up detection of unexpected services by **35%**
- Created Go script to prevent scanning blocked IPs, decreasing runtime by **20%** and complying with ISP requests

Projects

GeminiGo | github.com/HuangEric22/gemini_hackathon — Jan. 2026 – Present

- Built tool-augmented Gemini pipeline, grounding itinerary generation with Routes, Weather, and Restaurant data
- Reduced planning errors by injecting travel-time matrices, opening hours, and scheduling constraints into prompt
- Designed **multi-model inference routing** across Gemini Pro/Flash tiers to improve reliability under heavy load
- Utilized **Next.js, TypeScript, Drizzle, and Turso/SQLite** stack with caching to reduce latency and API calls

Spire Bot | github.com/HuangEric22/spire_bot — April 2026 – Present

- Architected a two-policy RL system for **Slay the Spire 2** with separate combat and planning agents
- Designed a **PPO-based PyTorch** combat policy over masked discrete actions and structured game-state features
- Built a simulation-data pipeline for trajectory generation, state featurization, and behavioral cloning warm-starts
- Utilized a **Python, PyTorch, NumPy, and .NET CLI** stack for rollouts, offline training, and agent evaluation

NoteLab | https://github.com/henrynvn09/NoteLab — March 2025 – April 2025

- Built note-taking tool in **TypeScript** with **Angular**, **FastAPI**, and **MongoDB**, containerizing with **Docker**
- Reduced transcription cost by **25%** by utilizing Deepgram API for real-time transcription with timestamped notes
- Boosted accuracy of scraped charts and diagrams by **35%** using metadata filtering and relevance scoring
- Engineered AI workflows using **chain-of-thought reasoning** to produce notes with images and citations

EMG2QWERTY | https://github.com/kevinhongca/emg2qwerty — Jan. 2024 – Mar. 2024

- Trained multiple neural architectures (CNN, LSTM, GRU, Transformer) to classify keystrokes from sEMG signals
- Improved Character Error Rate from 30% to 19.7% by using time separable convolutions and bidirectional LSTMs
- Identified Transformer failure cases as CER degraded to 45% due to weak fine-grained temporal EMG modeling
- Diagnosed robustness limits as cross-user inference degraded to 71% CER, exposing poor subject generalization

CenterPillars | https://github.com/HuangEric22/PointPillars — Sept. 2025 – Dec. 2025

- Trained 3D detection model by combining pillar BEV encoding with anchor-free, center-based detection heads
- Accelerated IoU and NMS calculations with custom CUDA kernels, reducing inference time from 16ms to 12ms
- Visualized model sensitivity with TensorBoard and Matplotlib to debug and validate localization improvements
- Achieved 68.20% BEV AP, a **6.75%** improvement over PointPillar baseline while maintaining low-latency inference